

Pet Territory Wars — Infrastructure & Deployment v1.0

Backend: Go

Database: PostgreSQL + PostGIS

Container: Docker

Deployment Model: Docker Compose

Architecture: Modular Monolith + Workers

Scope: MVP

1. Amaç

Bu doküman Pet Territory Wars MVP altyapısını tanımlar.

Temel hedefler:

- Düşük operasyon maliyeti
- Kolay deploy
- Kolay bakım
- Tek sunucuda çalışabilme
- API ve Worker'ları bağımsız ölçekleyebilme
- Binlerce eşzamanlı kullanıcıyı destekleyebilecek temel mimariyi oluşturmak

Bu doküman Kubernetes gibi büyük ölçekli orkestrasyon sistemlerini kapsamaz.

2. Temel Yaklaşım

MVP altyapısının felsefesi:

Basit başla, ihtiyaç oldukça büyüt.

İlk sürümde amaç;

- karmaşık sistem kurmak değil,
- oyunu piyasaya çıkarmaktır.

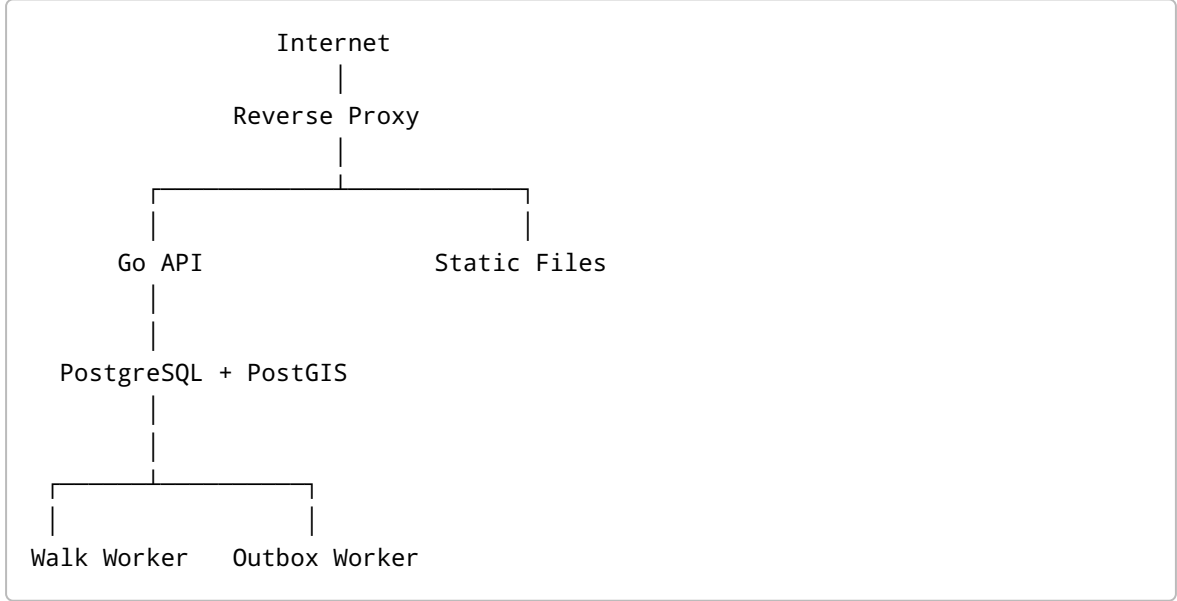
Bu nedenle:

```
Docker
+
Docker Compose
+
PostgreSQL
```

+
Go Services

yeterlidir.

3. Genel Mimari



Bütün servisler aynı Docker ağı içerisinde çalışır.

4. Servisler

MVP'de yalnızca aşağıdaki servisler bulunacaktır.

API

Görevi

- REST endpointleri
- Authentication
- Request validation
- Transaction yönetimi
- Application katmanı

API;

- Job çalıştırmaz
- Push göndermez
- Territory hesaplamaz

API stateless'tir.

İstenildiğinde birden fazla instance açılabilir.

Walk Worker

Görevi

Pending Walk

↓

Game Engine

↓

Database Update

Yalnızca

walk_processing_jobs

tablosunu dinler.

HTTP isteği almaz.

Stateless'tir.

İstenildiğinde çoğaltılabilir.

Outbox Worker

Görevi

Outbox Events

↓

Process

↓

Complete

MVP'de

analytics

cache invalidation

gelecek push sistemi

için hazırdır.

İlk sürümde iş yükü düşük olacaktır.

PostgreSQL

Tek source of truth.

İçerir:

Player

Walk

Territory

Score

Rules

Jobs

Outbox

5. Docker Yapısı

Önerilen yapı

docker /

api /

worker /

outbox /

docker-compose.yml

Her servis kendi Dockerfile'ına sahiptir.

6. Container Yapısı

```
api
↓
Go Binary
↓
Scratch veya Distroless Image
```

Worker için de aynı yapı kullanılır.

Küçük image tercih edilir.

7. Docker Compose

MVP

```
api
walk-worker
outbox-worker
postgres
reverse-proxy
```

servislerinden oluşur.

İleride

Redis

Prometheus

Grafana

eklenebilir.

8. Environment

Üç environment yeterlidir.

```
local  
  
staging  
  
production
```

Development için

Docker Compose

kullanılır.

9. Configuration

Config

Environment Variable

üzerinden okunur.

Örnek

```
HTTP_PORT  
  
DATABASE_URL  
  
LOG_LEVEL  
  
WORKER_CONCURRENCY  
  
MAX_BATCH_SIZE
```

Business Rule

config içerisinde tutulmaz.

Onlar RuleSet'tedir.

10. Secret Yönetimi

Secret'lar

repository içine girmez.

Örnek

```
Database Password  
  
JWT Secret  
  
Google OAuth Secret  
  
Apple Secret
```

Local

.env

kullanılabilir.

Production

Environment Variable

veya

hosting platformunun secret sistemi

kullanılır.

11. Database Connection Pool

API

Worker

Outbox

ayrı connection pool kullanır.

Tek pool paylaşılmaz.

Örnek

API

20 connection

Worker

10 connection

Outbox

5 connection

Bu değerler config'tir.

12. Migration

Migration

uygulama açılınca otomatik çalışmaz.

Deployment sırasında

ayrı migration komutu

çalıştırılır.

Sıra

Migration

↓

API

↓

Worker

13. Logging

Bütün servisler

stdout

üzerine JSON log yazar.

Örnek alanlar

```
trace_id  
walk_id  
player_id  
request_id  
job_id
```

Log dosyası uygulama tarafından yönetilmez.

Container runtime yönetir.

14. Health Check

Her servis

```
/health/live  
  
/health/ready
```

endpointine sahiptir.

Docker

healthcheck

bunları kullanır.

15. Worker Concurrency

Walk Worker

config üzerinden

kaç iş alacağını bilir.

Örnek

```
WORKER_CONCURRENCY=4
```

Bu sayı

CPU

Database

yüküne göre değiştirilebilir.

Kod değiştirmeden.

16. Job Queue

Harici Queue kullanılmaz.

Job Queue

PostgreSQL'dir.

Kaynak

```
walk_processing_jobs
```

Worker

```
SELECT ...  
FOR UPDATE SKIP LOCKED
```

mantığıyla

iş alır.

Bu MVP için yeterlidir.

17. Outbox

Outbox Pattern

kullanılır.

Kaynak

outbox_events

Worker

Pending event

okur.

İşler.

Completed yapar.

18. Redis

Redis

MVP'de zorunlu değildir.

Eklenmeyecektir.

Sebep

- Gereksiz operasyon
- Ek maliyet
- Cache ihtiyacı düşük

İleride

Leaderboard

Map cache

Session

için eklenebilir.

19. File Storage

MVP'de

Object Storage

kullanılmaz.

Çünkü

fotoğraf

video

dosya

yükleme yoktur.

20. Reverse Proxy

API

doğrudan internete açılmaz.

Önünde

Reverse Proxy

bulunur.

Görevleri

- HTTPS
 - Compression
 - Rate Limit
 - Forward Header
-

21. HTTPS

Production

HTTPS zorunludur.

HTTP

redirect edilir.

22. Backup

PostgreSQL

günlük backup alınmalıdır.

Öneri

Daily Backup

7 Gün

Production'da

Point In Time Recovery

destekleyen

managed PostgreSQL

tercih edilmelidir.

23. Restore

Backup

restore edilerek

ayrı ortamda

test edilmelidir.

Backup alınması

tek başına yeterli değildir.

24. Monitoring

MVP

minimum

CPU

Memory

Disk

Container Health

Database Connections

izlenmelidir.

Grafana

zorunlu değildir.

Hosting paneli yeterlidir.

25. Deploy

Deployment sırası

Migration

↓

API

↓

Worker

↓

Outbox Worker

Rollback gerekiyorsa

eski image

çalıştırılır.

26. CI

Her commit

Build

Test

Lint

çalıştırmalıdır.

Başarısız pipeline

deploy edilmez.

27. Ölçekleme

İlk sürüm

1 API

1 Walk Worker

1 Outbox Worker

1 PostgreSQL

Orta trafik

2 API

3 Worker

1 PostgreSQL

Yüksek trafik

4 API

8 Worker

Managed PostgreSQL

Kod değişmez.

Sadece

container sayısı artar.

28. Kubernetes

MVP'de

Kubernetes

KULLANILMAYACAKTIR.

Sebep

- Gereksiz operasyon yükü
- Gereksiz maliyet
- Gereksiz karmaşıklık
- MVP hedefiyle uyumsuz

Stateless API

ve

Stateless Worker

tasarımı sayesinde

ileride

Kubernetes'e

geçiş mümkündür.

Fakat

ilk sürümde

Docker Compose

yeterlidir.

29. Yüksek Trafik Senaryosu

Binlerce kullanıcı aynı anda

Walk Finish

gönderirse

API

yalnızca

Job oluşturur.

Asıl hesap

Worker'da yapılır.

Bu nedenle

API

çok kısa süre meşgul kalır.

İş yükü

Worker sayısı artırılarak

ölçeklenebilir.

30. Maliyet

MVP hedefi

minimum maliyet

maksimum ölçeklenebilirliktir.

İlk sürüm

tek sunucuda

çalışabilir.

Daha sonra

API

Worker

ayrı sunuculara taşınabilir.

Kod değişmez.

31. Disaster Recovery

Temel senaryolar

API çöker

↓

Container restart

Worker çöker

↓

İşler Database'de durur.

Yeni Worker devam eder.

Outbox çöker

↓

Event kaybolmaz.

Daha sonra işlenir.

Sunucu çöker

↓

Backup'tan geri dönülür.

32. Gelecekte Eklenebilecek Servisler

İhtiyaç oluştuğunda

Redis

Prometheus

Grafana

Object Storage

CDN

Push Notification Service

Kubernetes

eklenebilir.

Hiçbiri

MVP için zorunlu değildir.

33. Definition of Done

Infrastructure tamamlanmış sayılması için

- Docker Compose ile tek komut deploy edilebilmelidir.
 - API bağımsız container olmalıdır.
 - Walk Worker bağımsız container olmalıdır.
 - Outbox Worker bağımsız container olmalıdır.
 - PostgreSQL tek source of truth olmalıdır.
 - Migration ayrı çalışmalıdır.
 - Health check endpointleri bulunmalıdır.
 - HTTPS aktif olmalıdır.
 - Secret'lar repository'de bulunmamalıdır.
 - Günlük backup alınmalıdır.
 - Worker concurrency config üzerinden değiştirilebilmelidir.
 - Job Queue PostgreSQL üzerinden çalışmalıdır.
 - Kubernetes gerekmemelidir.
-

34. Nihai Karar

Pet Territory Wars MVP altyapısı aşağıdaki prensiplere göre kurulacaktır.

Docker tabanlı

Docker Compose ile yönetilen

Tek PostgreSQL kullanan

Stateless API

Stateless Worker

PostgreSQL Job Queue

Outbox Pattern

Düşük maliyetli

Kolay deploy edilebilir

Kolay ölçeklenebilir

Temel prensip:

MVP'nin amacı karmaşık bir altyapı kurmak değil, güvenilir ve kolay ölçeklenebilir bir ürün ortaya çıkarmaktır. Kubernetes ve benzeri büyük ölçek araçları ancak gerçek ihtiyaç doğduğunda değerlendirilecektir.